

Mini Report

Team 10, Week 9

Participating Team Members

Imogen: Broke up the Bank account god class in the “imogen” branch and documented the methodology.

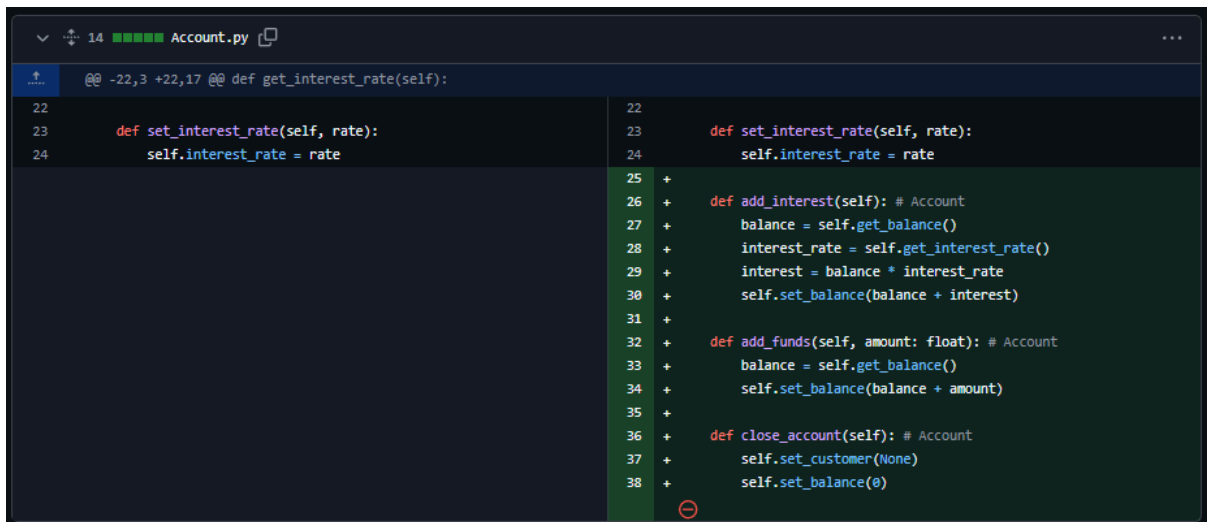
Amrutha: Removed some navigation code in the Account/Bank classes in the ‘amrutha’ branch, and used helper methods.

Anastasiia: Broke up the Bank god class in branch “nastya” and listed key changes for all classes in refactor_summary.md

We worked in individual branches and independently broke up the Bank god class. Our work is document in this pdf.

Move behaviour close to data

The add_interest, add_funds and close_account methods were delegated to the Account class, as shown below. This increased the cohesion within the Account and Bank class, while reducing navigation code.



```
Account.py
@@ -22,3 +22,17 @@ def get_interest_rate(self):
22
23     def set_interest_rate(self, rate):
24         self.interest_rate = rate
25 +
26 +     def add_interest(self): # Account
27 +         balance = self.get_balance()
28 +         interest_rate = self.get_interest_rate()
29 +         interest = balance * interest_rate
30 +         self.set_balance(balance + interest)
31 +
32 +     def add_funds(self, amount: float): # Account
33 +         balance = self.get_balance()
34 +         self.set_balance(balance + amount)
35 +
36 +     def close_account(self): # Account
37 +         self.set_customer(None)
38 +         self.set_balance(0)
```

Similarly, Branch, Customer, and Payroll methods were also delegated from the Bank class to their logically correct positions within their data classes, as shown below.

```
18 Branch.py
@@ -6,6 +6,7 @@ class Branch:
6     def __init__(self, location):
7         self._location = location
8         self._staff = []
9 +         self._opening_time = ""
10
11     def get_location(self):
12         return self._location
13
14 @@ -15,3 +16,20 @@ def set_location(self, location):
15
16     def get_staff(self):
17         return self._staff
18
19 +
20 +     def set_opening_time(self, opening_time):
21 +         self._opening_time = opening_time
22 +
23 +     def get_opening_time(self):
24 +         return self._opening_time
25 +
26 +     def close_branch(self, transfer_branch): # Branch
27 +         for staff in self.get_staff():
28 +             self.transfer_staff_member(transfer_branch, staff)
29 +
30 +     def transfer_staff_member(self, to_branch, staff: Staff): #
31 +         # Branch
32 +         self.get_staff().remove(staff)
33 +         to_branch.get_staff().append(staff)
34 +
35 +     def add_staff_member(self, staff: Staff): # Branch
36 +         self.get_staff().append(staff)
```

```
13 Customer.py
@@ -1,10 +1,23 @@
1 class Customer:
2     def __init__(self, name: str):
3         self._name = name
4
5     def set_name(self, name: str):
6         self._name = name
7
8     def get_name(self):
9         return self._name
10
11
12 class Customer:
13     def __init__(self, name: str):
14         self._name = name
15         self._address = ""
16         self._phone_number = ""
17
18     def set_name(self, name: str):
19         self._name = name
20
21     def get_name(self):
22         return self._name
23
24     def set_address(self, address: str):
25         self._address = address
26
27     def get_address(self):
28         return self._address
29
30     def set_phone_number(self, phone_number: str):
31         self._phone_number = phone_number
32
33     def get_phone_number(self):
34         return self._phone_number
```

```
5 Payroll.py
@@ -9,4 +9,7 @@ def get_staff_category_pay_schedule(self, staff_category):
9     return self.staff_category_pay_schedules[staff_category]
10
11     def get_staff_category_pay_day(self, staff_category):
12 -         return
13 +         self.staff_category_pay_schedules[staff_category].get_pay_date()
14
15
16     def get_staff_category_pay_schedule(self, staff_category):
17         return self.staff_category_pay_schedules[staff_category]
18
19     def get_staff_category_pay_day(self, staff_category):
20         return self.staff_category_pay_schedules[staff_category].get_pay_date()
21
22     def change_payroll_date(self, date: str, staff_category:
23         str): # maybe Payroll?
24
25     def get_staff_category_pay_schedule(self, staff_category):
26         return self.staff_category_pay_schedules[staff_category]
27
28     def set_pay_date(self, date):
29         self.get_staff_category_pay_schedule(staff_category).set_pay_date(
30             date)
```

Remove navigation code

Navigation code was removed in turn by moving behaviour into respective data classes. For example, the `change_payroll_date` method in the Bank god class previously required a long chain of calls:

`self.payroll.get_staff_category_pay_schedule(staff_category).set_pay_date(date)`

This was reduced to a simple `payroll.change_payroll_date(date, staff_category)` by moving the core payroll date changing logic into the Payroll class instead.

```
66 - self.payroll.get_staff_category_pay_schedule(staff_category).set_pay_date(date)
55 + payroll.change_payroll_date(date, staff_category)
```

Split up god classes

Splitting up of the Bank god class was primarily performed using the above two techniques:

Move behaviour close to data and **Remove navigation code**.

An additional technique was used to increase the cohesion of the Bank class and reducing unnecessary code. This required moving instance variables to other classes where appropriate.

Previously, the customer addresses, customer phone numbers, and branch opening times were stored in a dictionary in the Bank class, along with the customers and accounts lists. These were removed and placed as an instance variable within their respective classes, instead. This process has been highlighted in the below screenshots.

```
12 - self.customer_addresses = {} # key: customer, value:
    address
13 - self.customer_phone_numbers = {} # key: customer, value:
    phone number
14   self.branches = []
15 - self.branch_opening_times = {} # key: branch, value:
    opening time
```

Branch:

```
def __init__(self, location):
    self._location = location
    self._staff = []
    self._opening_time = ""
```

Customer:

```
def __init__(self, name: str):
    self._name = name
    self._address = ""
    self._phone_number = ""
```

Nastya: The relationship between classes was changed. A single bank has customers, branches and payroll. Each customer has a single name, address, phone number and multiple accounts. The Account class only stores the balance and interest.

```
class Customer:
    def __init__(self, name: str):
        self._name = name
        self.customer_address = "NO ADDRESS" # key: custo
        self.customer_phone_number = "NO PHONE NUMBER" #
        self.accounts = []

class Account:
    def __init__(self):
        self.customer = None
        self.balance = 0
        self.interest_rate = 0.05

class Bank:
    def __init__(self):
        self.accounts = []
        self.customers = []
        self.customer_addresses = {} # key: customer, value: address
        self.customer_phone_numbers = {} # key: customer, value: phone number
        self.branches = []
        self.branch_opening_times = {} # key: branch, value: opening time
        self.payroll = None
```

The customer gets removed from the customer list only after all possible accounts of the said customer have been deleted.

```
def close_account(self, customer: Customer, account: Account):
    customer.remove_account(account)

def change_opening_time(self, branch: Branch, time: str):
    self.branch_opening_times[branch] = time
    if customer.accounts:
        self.remove_customer(customer)
```

The customer will only be added once when creating a new account. For any further accounts, the customer will not be added.

```
def setup_new_account(self, account: Account, customer: Customer):
    account.set_customer(customer)
    self.accounts.append(account)
    customer.add_account(account)

    if customer not in self.customers:
        self.customers.append(customer)
        self.customer_addresses[customer] = "NO ADDRESS" # default add
        self.customer_phone_numbers[customer] = "NO PHONE NUMBER" # de
```